004-URL Parser

Difficulty: Medium Expect Time: 60 min Author: Wayne, Hsu

Uniform Resource Locator (URL) is a reference to a web resource that specifies its location on a computer network. For Example, `"https://www.ntust.edu.tw/home.php"` is a URL. A URL is a specific type of Uniform Resource Identifier (URI).

- **Basic Lexical Rule**
  - **ALPHA: Uppercase or lowercase English letter, ranging from A to Z or a to z**
  - **DIGIT: A single decimal digit character from 0 to 9**
  - **HEXDIGIT: DIGIT or a letter from A to F (uppercase or lowercase)**
  - **ALPHANUM: A character set composed of both ALPHA and DIGIT**
  - **UNRESERVED: A character that belongs to the ALPHANUM set, or is one of the following symbols: `-`, `.`, `_`, or `~`**
  - **GEN-DELIM = { ':', '/', '?', '#', '[', ']', '@' }**
  - **SUB-DELIM = {'!', '$', '&', '', '(', ')', '*', '+', ',', ';', '='}**
  - **RESERVED = A character set composed of both GEN-DELIM and SUB-DELIM.**

- **URL Encoding**

  Percent-encoding is a way to represent characters in a URI using only ASCII. Each encoded sequence starts with `%` followed by two **HEXDIGIT** characters. When encountering **Percent-encoding**, treat the two characters after `%` as a hexadecimal number and look up the corresponding ASCII character.

  Example:

  I. **%20** will parsed **0x20**, **ASCII(0x20)** is `  `(space)

  II. **%21** will parsed **0x21**, **ASCII(0x21)** is `!`

(space)	!	"	#	$	%	&	'	(	)	*
%20	%21	%22	%23	%24	%25	%26	%27	%28	%29	%2A
+	,	/	:	;	=	?	@	[	]	
%2B	%2C	%2F	%3A	%3B	%3D	%3F	%40	%5B	%5D	

Table of percent-encoded representations for **RESERVED** characters.

- **URL Structure**

  `[Protocol]://[Authority][Pathname]?[Query]#[Fragment]`

  - **Protocol**: Composed of **ALPHA**.

    Example:

    I. https

    II. ftp

  - **Authority**:

    `[Username ":" Password "@"] Hostname [ ":" Port ]`

    - Username<optional>:

      Starts with an **ALPHA** , followed by zero or more **ALPHANUM**.
    - Password <optional>:

      Composed only of **ALPHANUM**. Must be present only if a Username is provided.
    - Hostname:

      One or more labels separated by `.`. Each label is composed of **UNRESERVED** and `-`. The final character of the hostname must be an **ALPHANUM**.
    - Port<optional>:

      A **DIGIT** sequence, in the range 0 to 65535.

    If the username and password are present, they must appear as a pair separated by a colon (`:`) and be followed by an at symbol (`@`) before the hostname; additionally, if the port is present, it must follow the hostname and be prefixed with a colon (`:`).

    Example:

    I. `zh.wikipedia.org`

    II. `john.doe:abc123@www.example.com:1234`

    III. `api.gateway.io:443`

  - **Pathname**

    A pathname is a sequence of segments separated by slashes (`/`), and starts with `/`. Each segment may contain **UNRESERVED** and **RESERVED**, but **NOT INCLUDE** the delimiters `?`, `#`, `/`. The segments may be empty string, but pathname must contain at least one non-empty string segment.

    Example:

    I. `/forum/questions/`

    parsed as **vector{"forum", "questions"}**

    II. `/api/test`

    parsed as **vector{"api", "test"}**

    III. `/api////test`

    parsed as **vector{"api", "test"}**

    IV. `/data//////`

    parsed as **vector{"data"}**

- **Querystring<optional>**: Appears after the first `?` character
  `Key "=" Value [ "&" Key "=" Value ]*`
  - Key: Composed of **ALPHANUM** characters
  - Value: include **UNRESERVED** and **RESERVED** sequences, but **NOT INCLUDE** the delimiters `&`, `#`, `=`
  - Multiple queries are separated by `&`.

  Example:
  I. `tag=networking&order=newest`
  II. `boardType=0&searchType=&searchText=&Page=4`
  III. `tag=4&tag=5`

  Querystring allows **DUPLICATE keys**. If duplicate keys are present, they should be parsed into a **`std::vector`**.

```
using KeyType = std::string
using ValueType = std::vector<std::string>
using Query = std::map<KeyType, ValueType>

Query queries = {{"Key", {"1", "2", "3"}}};
```

If the querystring **Key=1&Key=2&Key=3** appears, it should be parsed into the same type as shown in the figure.

- **Fragment <optional>**: Any string after the first `#` character.

  Example:
  I. `#top`
  II. `#:~:text=whatever`

- **Requirements**
  1. Implement the method:
     `static std::shared_ptr<URL> from(std::string inputUrl)`.
  2. All members are allowed to be modified, and after any modification, **`location()`** must return the updated URL.
     a) The test case will **NOT** modify the **Querystring**.
     b) The test case will **NOT** modify any **Pathname** segments to be empty strings.
  3. Implement the method **`std::string location()`** to reconstruct the URL based on object attributes.
     a) Trailing `/` in ***Pathname*** must be trimmed
     b) Trailing `&` in ***Query*** must be trimmed.
     c) Query keys must be sorted **alphabetically**.
     d) If either the **`username`** or the **`password`** is an empty string,
        DO NOT display the **`[username ":" password "@"]`** segment.
     e) If the port is **`0`**, **DO NOT** display the the **`[ ":" Port ]`** segment.
     f) When an optional section is missing, its delimiter should not be output either.
        i. if **Querystring** is empty, `?` **DO NOT** display
        ii. if **Fragment** is an empty string, `#` **DO NOT** dispaly

```
https://yanhau:passwd@yhchen.org:80/path/to/test/?tag=value1p&tag2=newest#top1564aegr
└┬┘   └────────────────────────┘└────────────┘└──────────────────────┘└──────────┘
Protocol          authority            path              query            fragment
```

Input

1. Complete the required class implementation. The following files already contain some implementations
   a) `URL.h`
   You can submit a header-only implementation, or split the implementation into **`*.cpp`** files.

2. The Online Judge system will replace the following files:
   a) `entrypoint.cpp`
   b) `case.h`
   c) `test.h`

3. The test cases for student available in **case.h**, but not all test data is included in that file. The Online Judge system will also include hidden test inputs.

4. Input range
   a) The input will be a **Single percent-encoded** URL string, For example:
   https%3A%2F%2Fen.wikipedia.org%2Fwiki%2FUniform_Resource_Identifier
   which should be decoded to:
   https://en.wikipedia.org/wiki/Uniform_Resource_Identifier
   b) Consider the following URL:
   https%253A%252F%252Fen.wikipedia.org%252Fwiki%252FUniform_Resource_Identifier%2525
   which should be decoded to:
   https%3A%2F%2Fen.wikipedia.org%2Fwiki%2FUniform_Resource_Identifier%25
   Such a URL will **NOT** appear, as it requires decode twice.
   c) All decoded URLs conform to the specified format, so you should not spend time verifying whether the URL matches the description.
   d) The following cases will not occur:
      i. Inputs with unpaired or incomplete query strings (e.g., **?key**, **?key=**) will not be included.
      ii. URLs with an entirely empty pathname (such as http://en.wikipedia.org/) are excluded.

Output

Online Judge will run the test:

1. **DON' T** output any information to the terminal.
2. When any test failed, Online Judge will try to list error messages.

Note

Simplified URL Grammar based on RFC 2234 / RFC 5234 / RFC 3986

URL ::= protocol "://" authority pathname ["?" query] ["#" fragment]
A URL consists of protocol, authority, path, optional query, and optional fragment.

protocol ::= **[A-Za-z]+**
Protocol must consist of only alphabetic letters (e.g., "http", "https").

authority ::= [username ":" password "@"] hostname [":" port]
Authority may include optional username/password, followed by hostname and optional port.

username ::= **[A-Za-z]+**
Username must start with a letter and contain only letters and hyphens.

password ::= **[A-Za-z0-9]+**
Password appears only when username is present.

hostname ::= label ("." label)*, ends with a letter
Hostname is made of one or more labels separated by dots, and ends with a letter.

label ::= **[A-Za-z0-9.~-_]+**
A label is a non-empty sequence of letters or hyphens.

port ::= 0 ~ 65535
Port is a positive integer.

pathname ::= "/" segment ("/" segment)* ["/"]?
Path consists of segments separated by "**/**", may end with an optional trailing slash.

segment ::= **[A-Za-z0-9\-._~!$&'()*+,;=:@%]***
Each segment may contain reserved, unreserved, and percent-encoded characters.
Percent-encoded format: %XX, where X is a hexadecimal digit.

query ::= pair ("&" pair)*
Query is composed of key=value pairs separated by '**&**'.

```
pair        ::= key "=" value
# Each key-value pair connects a query parameter to a value.

key         ::= [A-Za-z0-9]+
# Keys use only letters and hyphens.

value       ::= [A-Za-z0-9\-._~!$'()*+,;:@/%]*
# Values may include reserved, unreserved, and percent-encoded characters.

fragment    ::= any characters after '#'
# Fragment is optional and accepts any content after '#' until the end of the URL.

pct-encoded ::= "%" HEXDIG HEXDIG
# A percent-encoded sequence begins with '%' followed by two hexadecimal digits.
# It represents a single byte using its ASCII hexadecimal form.
# Example: "%20" represents a space character(ASCII 0x20).

HEXDIG ::= [0-9A-Fa-f]
# A single hexadecimal digit (0–9, A–F, or a–f).
```